

ANALISA LOAD BALANCING PADA CLOUD COMPUTING MENGGUNAKAN METODE LEAST CONNECTION

Abdurrahman Hafiz¹,

¹Sistem Komputer

²Fakultas Ilmu Komputer

³Universitas Sriwijaya

Penulis Korespondensi: Abdurrahman Hafiz (hafizsyahab2582@gmail.com)

Diterima	:	01 Januari 2022	URL	:	https://jik.netplg.com/index.php/open/article/view/2
Disetujui	:	10 Januari 2022	DOI	:	https://doi.org/nomordoi
Diterbitkan	:	30 Januari 2022	ISSN	:	234-2341

ABSTRAK

Cloud computing menyediakan sumber daya komputasi secara dinamis melalui jaringan, namun peningkatan jumlah permintaan pengguna dapat menyebabkan beban server meningkat dan menurunkan kinerja layanan. Penelitian ini bertujuan menganalisis performa load balancing menggunakan metode Least Connection dengan HAProxy sebagai load balancer. Implementasi dilakukan pada lingkungan virtual machine berbasis VMware yang terdiri dari satu client, satu load balancer, dan tiga server backend Apache. Pengujian dilakukan menggunakan Apache JMeter dengan variasi jumlah pengguna secara bertahap. Parameter Quality of Service (QoS) yang dianalisis meliputi throughput, delay, packet loss, jitter, error rate, dan response time. Hasil pengujian menunjukkan bahwa metode Least Connection mampu mendistribusikan beban berdasarkan koneksi aktif sehingga trafik lebih merata. Meskipun terjadi peningkatan delay dan response time pada beban tinggi, sistem tetap berjalan stabil dengan tingkat error rendah. Metode ini dinilai efektif dalam meningkatkan kinerja dan keandalan layanan cloud computing.

Kata kunci: Cloud Computing, Load Balancing, Least Connection, HAProxy, Quality of Service (QoS)

ABSTRACT

Cloud computing provides dynamic computing resources over a network. However, an increasing number of user requests can lead to higher server workloads and decreased service performance. This study aims to analyze the performance of load balancing using the Least Connection method with HAProxy as the load balancer. The system was implemented in a VMware-based virtual machine environment consisting of one client, one load balancer, and three Apache backend servers. Performance testing was conducted using Apache JMeter with gradually increasing numbers of users. The Quality of Service (QoS) parameters analyzed include throughput, delay, packet loss, jitter, error rate, and response time. The results show that the Least Connection method effectively distributes traffic based on active connections, resulting in more balanced load distribution. Although delay and response time increased under higher workloads, the system remained stable with a low error rate. Therefore, this method is considered effective in improving the performance and reliability of cloud computing services.

Keywords: Cloud Computing, Load Balancing, Least Connection, HAProxy, Quality of Service (QoS)

1. PENDAHULUAN

1.1 Latar Belakang

Cloud computing merupakan sebuah gabungan pemanfaatan teknologi komputer dan pengembangan internet yang mengalami perkembangan sangat pesat dalam beberapa tahun terakhir. Menurut laporan Gartner, sebuah perusahaan riset global yang menyediakan informasi berbagai bidang salah satunya dalam bidang teknologi informasi menyatakan bahwa “Cloud akan membawa perubahan pada industri teknologi informasi, cloud computing juga akan mengubah hidup kita dengan menyediakan jenis layanan baru kepada pengguna”[1]. Pengguna tersebut akan menggunakan layanan dari cloud dengan tidak memperhatikan detailnya.

Cloud computing juga memberikan kemudahan dalam menyimpan, mengakses dan mengolah beberapa informasi. Dengan kenyamanan tersebut cloud computing dapat memungkinkan akses jaringan di mana-mana, sesuai permintaan ke beberapa sumber daya komputasi yang dapat dikonfigurasi seperti, jaringan, server, penyimpanan, aplikasi dan layanan. Cloud computing itu sendiri terbagi menjadi 3 jenis, yang pertama ialah Infrastructure as a Service (IaaS), yang kedua ialah Software as a Service (SaaS), dan Platform as a Service (PaaS) [2].

Saat ini dengan meningkatnya permintaan kebutuhan informasi ke dalam sumber daya komputasi terutama pada server, menyebabkan jalannya beban traffic pada sebuah server semakin besar. Dengan semakin besarnya beban traffic yang terjadi pada sebuah server menyebabkan terjadinya kelebihan beban kerja server. Untuk dapat membantu mengurangi permasalahan beban kerja yang berlebihan pada suatu server, maka digunakanlah sebuah metode yang ada pada cloud computing yaitu metode load balancing.

Load balancing merupakan metode pada jaringan komputer yang diartikan menjadi sebuah penyeimbang beban. Cara kerja load balancing yaitu dengan mendistribusikan beban traffic yang ada pada sumber daya komputasi dengan melakukan pembagian beban kerja pada dua jalur atau lebih pada sebuah sumber daya komputasi dengan merata. Proses tersebut dilakukan untuk dapat membuat sebuah sumber daya komputasi dapat bekerja dengan lebih cepat dan ringan.

Ada dua tipe perangkat yang dapat digunakan dalam membantu load balancing yaitu software load balancing dan hardware load balancing. Software load balancing dimana load balancing berjalan pada sebuah perangkat lunak yang dapat di install dan dikonfigurasi sebelum digunakan pada sebuah komputasi atau server. Dan hardware load balancing di mana load balancing berjalan pada sebuah perangkat keras yang sudah siap digunakan [3].

Pada load balancing terdapat beragam metode algoritma yang melakukan load balancing salah satunya adalah algoritma Least Connection. Least Connection merupakan algoritma pada load balancing yang bekerja dengan melakukan pembagian beban kerja pada server berdasarkan koneksi aktif dengan mengarahkan secara langsung pada koneksi yang memiliki jumlah koneksi paling sedikit. Algoritma ini dapat digunakan pada lingkungan server yang berubah-ubah [4].

Penelitian mengenai load balancing dengan judul “A Load Balancing Model Based on Cloud Partitioning for the Public Cloud”. Penelitian tersebut membahas mengenai model load balancing berbasis partisi cloud pada cloud public. Model tersebut membagi cloud menjadi partisi, tujuannya untuk mendistribusikan beban kerja dari masing-masing partisi. Meskipun cukup inovatif, namun penelitian ini memiliki kelemahan yaitu tidak menggunakan salah satu metode load balancing, tetapi pendekatan algoritma sebagai solusi dalam menangani load balancing [1].

Penelitian lain dengan judul “Load Balancing Analysis Using Round-Robin and Least-Connection Algorithms for Server Service Response Time”. Penelitian ini membahas mengenai perbandingan response time antara metode Round Robin dan Least Connection. Penelitian ini menunjukkan bahwa koneksi yang didistribusikan ke masing-masing server dengan 100 permintaan pada 500 koneksi per detik dan 1200 permintaan pada 600 koneksi per detik, dengan melakukannya pada kedua metode algoritma load balancing. Hasilnya metode Round Robin lebih unggul dalam response time dibandingkan metode Least Connection. Kekurangan penelitian ini hanya berfokus kepada response time dibandingkan parameter lain serta dalam skala kecil [5].

Penelitian lain dengan judul “An Improved Weighted Least Connection Scheduling Algorithm for Load Balancing in Web Cluster Systems”. Hasil dari penelitian ini didapatkan penjagaan terhadap keseimbangan muatan di antara server dengan menghindari kelebihan beban pada server baru yang ditambahkan. Kelemahan penelitian ini tanpa menggunakan tool load balancing dan tanpa uji cloud environment [6].

Pada penelitian yang lain dengan judul “Analisa Implementasi Load Balancing Round Robin dan Least Connection Pada Web Server”. Penelitian ini melakukan analisa implementasi metode Round Robin dan Least Connection. Dari hasil penelitian pengujian menunjukkan bahwa metode Least Connection lebih baik dalam menyeimbangkan beban saat koneksi meningkat. Namun penelitian ini memiliki kekurangan seperti cakupan pengujian yang terbatas, tidak adanya variasi jumlah user yang digunakan, serta tidak adanya penggunaan alat benchmark. Selain itu penelitian hanya terbatas pada pengujian dasar dan tidak memberikan kinerja performa sistem [4].

Pada penelitian yang lain dengan judul “An Improvement on the Weighted Least-Connection Scheduling Algorithm for Load Balancing in Web Cluster Systems”. Penelitian ini melakukan peningkatan algoritma Weighted Least Connection pada web server, jika permintaan web ditugaskan ke server lebih dari L kali hasilnya algoritma yang ditingkatkan dapat menjaga keseimbangan beban antara server yang dengan menghindari kelebihan beban saat penambahan server yang baru. Kekurangan penelitian ini tidak melakukan pengujian performa load balancing dan hanya fokus pada efisiensi dari algoritma yang digunakan [7].

Berdasarkan penjelasan yang telah diuraikan diatas, maka penulis akan melakukan penelitian dengan judul “ANALISA LOAD BALANCING PADA CLOUD COMPUTING DENGAN MENGGUNAKAN METODE LEAST CONNECTION”.

1.2 Rumusan Masalah

Rumusan masalah pada penelitian ini adalah :

1. Bagaimana menganalisis kinerja server yang dengan cara mendistribusikan beban kerja server ?
2. Bagaimana implementasi load balancing pada cloud computing dengan menggunakan algoritma least connection ?
3. Bagaimana mengukur performa kinerja Quality of Service (QoS) pada load balancing dengan menggunakan algoritma least connection.

1.3 Tujuan

Adapun tujuan yang hendak dicapai dari penelitian ini adalah :

1. Mampu menganalisis Quality of Service (QoS) pada load balancing dengan menggunakan metode least connection.
2. Memastikan kinerja server menjadi lebih stabil setelah melakukan load balancing pada server.
3. Mampu melakukan implementasi load balancing pada cloud computing dengan menggunakan algoritma least connection.

1.4 Manfaat

Adapun manfaat yang diharapkan dapat diambil dari penelitian ini adalah :

1. Dapat membantu mengoptimalkan penggunaan sumber daya server.
2. Dapat meningkatkan kemampuan kinerja sebuah server agar menjadi lebih cepat dan ringan.
3. Dapat membantu mengurangi terjadinya kelebihan beban kerja pada server.

1.5 Batasan Masalah

Adapun batasan masalah pada penelitian ini antara lain :

1. Menggunakan virtualisasi server yaitu Linux Ubuntu sebagai server utama.
2. Load balancer yang digunakan adalah jenis load balancer berbasis software yaitu HAProxy.
3. Menggunakan dua atau lebih server backend.

2. TINJAUAN PUSTAKA

2.1 Cloud Computing

Cloud computing adalah teknologi yang menggunakan sebuah perangkat komputasi melalui internet. Cloud computing dapat memungkinkan akses sebuah sumber daya komputasi di mana pun selagi terhubung dengan internet, sumber daya komputasi tersebut seperti jaringan, server penyimpanan, aplikasi dan layanan [2].

Cloud computing juga tidak hanya menjadi teknologi modern tetapi juga telah berkembang menjadi sebuah fondasi utama dalam pengelolaan sumber daya komputasi. Cloud computing dapat menggabungkan berbagai sumber daya seperti pemrosesan, penyimpanan, dan jaringan ke dalam satu lingkungan yang terintegrasi dan dapat diakses secara luas.

Cloud computing memiliki beberapa karakteristik utama beberapa diantaranya yaitu on-demand self-service, yaitu kemampuan pengguna untuk menyediakan dan mengatur sumber daya komputasi secara mandiri tanpa interaksi langsung dengan pihak penyedia layanan.

Selain itu karakteristik lainnya dari cloud computing yaitu broad network access yang di mana dapat memungkinkan layanan cloud diakses melalui berbagai perangkat seperti komputer, laptop, maupun gadget lainnya selagi terhubung dengan internet. Lalu ada Resource pooling, yaitu penggabungan seluruh sumber daya komputasi ke dalam satu kumpulan besar untuk melayani berbagai pengguna secara bersamaan.

Karakteristik lainnya yaitu kemampuan rapid elasticity di mana sumber daya komputasi dapat diperbesar dan diperkecil secara cepat sesuai dengan beban kerja dari cloud computing tersebut. Yang terakhir dari karakteristik cloud computing yaitu measured service, yaitu sistem pengukuran otomatis terhadap penggunaan sumber daya hingga memungkinkan penerapan model pembayaran pay-as-you-go. Dengan demikian, pengguna akan dikenakan biaya sesuai konsumsi layanan.

Cloud computing itu sendiri dibangun oleh dua komponen utama, yaitu front-end dan back-end. Komponen front-end terdiri dari perangkat, antarmuka, dan aplikasi yang dapat digunakan pengguna untuk mengakses layanan cloud. Sementara itu, komponen back-end terdiri atas keseluruhan infrastruktur layanan cloud,

server, sistem penyimpanan, jaringan, hypervisor, dan mekanisme orkestrasi layanan. Pada bagian ini semua proses komputasi akan dijalankan dan dikelola, sehingga pengguna tidak perlu mengetahui detail teknis dari proses yang berlangsung.

Cloud computing memiliki sejumlah kelebihan yang menjadikannya teknologi yang banyak diadopsi oleh organisasi maupun pengguna individu. Kelebihan utama cloud computing adalah efisiensi biaya dikarenakan pengguna tidak perlu melakukan investasi perangkat keras secara langsung, melainkan cukup membayar sesuai penggunaan layanan.

Selain itu, cloud computing menawarkan fleksibilitas tinggi, di mana sumber daya komputasi dapat ditingkatkan atau dikurangi dengan cepat sesuai kebutuhan. Cloud juga mendukung aksesibilitas yang sangat luas, hal tersebut memungkinkan pengguna dapat bekerja dari mana saja selama terhubung dengan internet. Tidak hanya itu, penyedia layanan cloud umumnya menyediakan sistem keamanan berlapis, redundansi data, serta Tingkat ketersediaan layanan yang tinggi, sehingga dapat memastikan operasional layanan tetap layanan stabil.

Cloud computing juga memiliki beberapa kekurangan yang perlu diperhatikan. Ketergantungan penuh terhadap koneksi internet menjadi tantangan utama, dikarenakan kualitas akses layanan sangat dipengaruhi oleh stabilitas jaringan. Selain itu, adanya isu keamanan dan privasi yang menjadi perhatian, terutama bagi organisasi yang menangani data sensitif, karena seluruh data dikelola oleh pihak penyedia layanan. Dan adanya risiko vendor lock ini, yaitu kesulitan migrasi dari satu penyedia cloud ke penyedia lain akibat perbedaan struktur layanan atau konfigurasi sistem. Dalam beberapa kasus, biaya cloud dapat menjadi tinggi apabila penggunaan sumber daya tidak dikelola dengan baik, terutama pada skala besar atau beban kerja yang tidak stabil.

Cloud computing memiliki layanan utama yang dibagi atas tiga layanan yaitu Software as a Service (SaaS), Platform as a Service (PaaS), dan Infrastructure as a Service (IaaS). Layanan tersebut merupakan kumpulan layanan pada cloud computing yang ditampilkan dalam bentuk piramida sebagai arsitektur cloud computing.

2.2 Metode Least Connection

Metode Least Connection adalah salah satu dari algoritma load balancing dynamic, metode ini akan melakukan pemeriksaan server yang memiliki koneksi paling sedikit dan mengirimkan permintaan ke server tersebut [5]. Pada Least Connection ini server backend akan dianggap memiliki kinerja yang sama, algoritma ini dapat dijalankan dengan menggunakan perintah `balance leastconn` pada server backend dan juga konfigurasi pada backend setting. Least Connection sangat cocok untuk digunakan pada server dynamic yang sering mengalami perubahan.

Selain itu Least Connection juga sangat sesuai bagi server yang sering memiliki waktu koneksi yang berbeda beda. Selain itu, metode Least Connection juga memberikan performa yang stabil pada sistem dengan trafik yang sulit diprediksi. Ketika suatu server mengalami peningkatan beban secara tiba-tiba, algoritma ini secara otomatis mengalihkan permintaan baru ke server lain yang lebih ringan tanpa memerlukan konfigurasi tambahan. Kemampuan adaptif ini membuat Least Connection sering digunakan pada layanan berbasis web, API, maupun aplikasi real-time yang memerlukan respons cepat dan konsisten. Dengan penerapan yang tepat, metode ini dapat meningkatkan efisiensi pemanfaatan sumber daya serta mengurangi risiko terjadinya bottleneck pada salah satu server.

2.3 High Availability Proxy

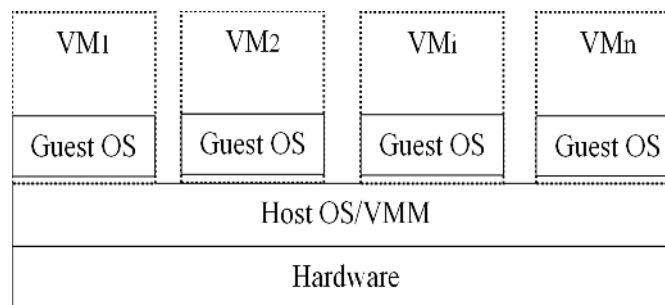
High Availability Proxy atau HAProxy adalah sebuah software berbasis yang dapat membantu proses load balancing pada cloud computing. HAProxy hanya dapat dijalankan dengan menggunakan sistem operasi Linux dan bersifat open source. Metode algoritma dalam load balancing yang dapat digunakan pada HAProxy seperti, Round Robin, Weighted Round Robin, Least Connection, Weighted Least Connection, dan IP Hash

HAProxy sebagai load balancer dapat bekerja pada aplikasi berbasis HTTP atau TCP. Tujuan dari HAProxy itu sendiri untuk mendistribusikan beban kerja yang ada pada server ke server lain dan meningkatkan kinerja server. Cara kerja-Nya yaitu dengan HAProxy mengirimkan ke frontend sebagai penerima traffic kemudian mengirimkan ke server backend, lalu backend sebagai penerima semua permintaan akan mengirimkan kembali kepada HAProxy dan diterima oleh klien [8].

Selain fungsi utamanya sebagai load balancer, HAProxy juga menawarkan berbagai fitur pendukung yang meningkatkan keandalan layanan, seperti health check untuk memantau kondisi setiap server backend, logging untuk analisis trafik, serta kemampuan failover otomatis ketika terdapat server yang tidak merespons. Dengan adanya mekanisme ini, sistem dapat berjalan meskipun salah satu server mengalami gangguan. HAProxy juga mendukung konfigurasi fleksibel yang memungkinkan administrator melakukan pembatasan koneksi, pengaturan timeout, serta penerapan SSL termination sehingga proses pengamanan data dapat dilakukan secara lebih efisien. Fitur-fitur tersebut menjadikan HAProxy salah satu solusi load balancing yang paling banyak digunakan pada lingkungan produksi berskala kecil hingga besar.

2.4 Virtual Machine

Virtual Machine (VM) Virtual Machine (VM) merupakan sebuah virtualisasi dalam sistem komputer yang ditemukan pada tahun 1960 oleh IBM dengan versi pertama yaitu CP-40. Virtual machine itu sendiri adalah perangkat lunak dalam bentuk virtual yang dapat menjalankan sebuah program sistem operasi dengan dengan mengemulasi perangkat keras komputer yang terdiri dari CPU, memori, melakukan pemeliharaan sistem sendiri. Sehingga bisa memiliki sebuah perangkat fisik pada setiap virtualisasi [9].



Gambar 2.1 Arsitektur Virtual Machine

Pada Gambar 2.1 menunjukkan arsitektur Virtual Machine, pada lapisan paling bawah di atas hardware terdapat kernel Host OS atau Virtual Machine Monitor (VMM) yang berfungsi menyediakan sumber daya untuk virtual machine dengan setiap virtual machine, beberapa layanan berjalan di atas Guest OS yang menyediakan serangkaian abstraksi tingkat tinggi seperti akses file dan dukungan jaringan ke aplikasi yang berjalan pada virtual machine.

Virtual Machine dapat memberikan kemudahan dalam hal distribusi sumber daya perangkat keras yang ada pada tamu, dengan menjalankan sistem operasi tamu itu sendiri. Sebuah perangkat lunak yang menyediakan virtualisasi disebut sebagai virtual machine atau hypervisor. Keuntungan utama dari sistem virtual machine adalah.

1. Dapat berbagi sistem operasi yang dijalankan pada komputer yang sama
2. Virtual machine menyediakan arsitektur set yang berbeda dengan perangkat keras.

Selain itu virtual machine juga mempunyai keunggulan lain dalam mengembangkan sistem operasi yaitu dalam akses debugging yang lebih baik dan reboot lebih cepat dibandingkan yang lain. Dalam virtual machine terdapat alat virtualisasi yang sering dipakai sebagai proses virtualisasi. Alat tersebut bersifat open source dan juga gratis, ini merupakan beberapa alat virtualisasi yang paling banyak digunakan yaitu sebagai berikut.

2.5 Ubuntu

Ubuntu adalah sistem operasi dari distribusi linux yang bersifat open source yang dikeluarkan pertama kali pada tahun 2004. Ubuntu merupakan sistem operasi yang dibangun berdasarkan pada sistem operasi berbasis linux yaitu Debian, sehingga hampir beberapa fitur yang ada pada Ubuntu sangat mirip dengan Debian [10]. Ubuntu versi terbaru saat ini untuk long-term support dengan Ubuntu versi 22.04.3 LTS dengan nama kode 'Jammy Jellyfish' dalam bentuk desktop, server dan sementara itu untuk versi terbaru yang baru dirilis oleh Ubuntu yaitu, Ubuntu versi 23.10 dengan nama kode 'Mantic Minotaur'namu belum dalam status LTS.

Salah satu kelebihan dari Ubuntu daripada sistem operasi linux yang lain adalah Ubuntu lebih ringan dalam menggunakan memori untuk beroperasi, sehingga Ubuntu akan lebih cepat dan lancar dalam kinerjanya. Selain itu Ubuntu juga menawarkan kemudahan dalam hal pembaruan dan manajemen paket melalui sistem manajemen paket APT (Advanced Package Tool). Sistem ini memungkinkan pengguna untuk menginstal, memperbarui, dan menghapus perangkat lunak dengan cepat dan aman, sehingga meminimalkan risiko konflik atau masalah kompatibilitas. Fitur ini menjadikan Ubuntu populer baik untuk pengguna desktop maupun server, karena selain ringan dan cepat, sistem ini juga relatif mudah untuk dikelola bahkan oleh pengguna yang baru mengenal linux.

2.6 Quality of Service (QoS)

MenurutQuality of Service (QoS) adalah mekanisme yang ada pada jaringan untuk mengelola kualitas transmisi lalu lintas atau traffic jaringan dan mengukur kinerja output yang dihasilkan [11]. Quality of Service pada umumnya memiliki tiga atribut yang dapat digunakan untuk mengukur output dari sebuah proses yaitu,

ketepatan waktu (timeliness) digunakan untuk mengukur waktu yang dibutuhkan untuk menghasilkan output, presisi (precision) digunakan untuk mengukur jumlah atau kuantitas dari output yang dihasilkan, dan akurasi (accuracy) digunakan untuk mengukur ketepatan output yang dihasilkan. Pada jaringan komputer terdapat parameter Quality of Service yang dapat digunakan diantara-Nya yaitu, Delay, Jitter, Packet Loss, dan Throughput.

Parameter Quality of Service akan digunakan sebagai salah satu metode analisa kinerja load balancing pada web server dengan menggunakan metode Least Connection. Berikut persamaan perhitungan atau rumus dari parameter Quality of Service.

- Throughput (bps) = $\frac{\text{Total Data (Byte)} \times 8}{\text{Durasi Pengujian (s)}}$
- Delay = $\frac{\text{Total Delay}}{\text{Total paket yang diterima}}$
- Packet Loss = $\frac{(\text{Paket yang dikirim} - \text{Paket yang diterima}) \times 100\%}{\text{Paket data yang dikirim}}$
- Jitter = $\frac{\text{Total variasi Delay}}{\text{Total paket yang diterima}}$

Quality of Service juga berperan penting dalam memastikan bahwa layanan jaringan dapat berjalan stabil meskipun terjadi peningkatan beban lalu lintas. Dengan melakukan pengukuran QoS, administrator jaringan dapat mengetahui apakah proses distribusi beban oleh load balancer bekerja secara optimal atau tidak. Parameter-parameter tersebut membantu mengidentifikasi bottleneck, mengevaluasi kemampuan server dalam menangani permintaan secara bersamaan, serta memastikan pengalaman pengguna tetap baik. Analisis QoS juga memungkinkan dilakukan optimasi konfigurasi seperti penyesuaian algoritma load balancing, pengaturan jaringan, serta peningkatan kapasitas server agar performa keseluruhan sistem tetap terjaga.

2.7 Apache JMeter

Apache JMeter adalah alat pengujian load balancing menggunakan Bahasa pemrograman Java. Apache JMeter dapat digunakan untuk menganalisis dan mengukur kinerja dari berbagai layanan berbasis web server, seperti HTTP dan HTTPS. Apache JMeter juga merupakan sofwater berbasis open source, versi terbaru Apache JMeter yaitu 5.6.3 yang dirilis pada tahun 2024 dengan menggunakan Bahasa pemrograman Java 8+.

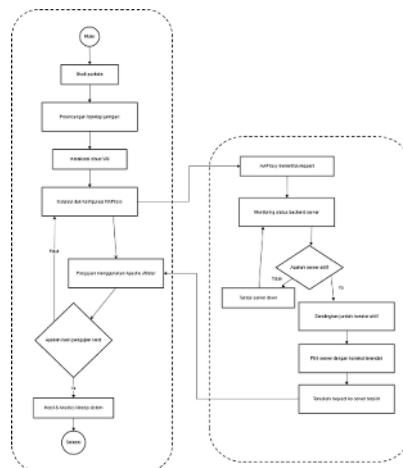
Selain itu, Apache JMeter menyediakan antarmuka pengguna yang intuitif dan mendukung berbagai jenis pengujian, termasuk load testing, stress testing, dan performance testing. Dengan kemampuan untuk menyimulasikan sejumlah besar pengguna secara bersamaan, JMeter memungkinkan pengembang dan administrator sistem untuk mengidentifikasi bottleneck, memantau respons server, serta mengevaluasi kinerja aplikasi web secara menyeluruh. Dukungan terhadap protokol populer seperti HTTP, HTTPS, FTP, JDBC, dan lainnya membuat Jmeter fleksibel dan dapat diterapkan pada berbagai skenario pengujian di lingkungan server maupun cloud [12].

3. METODELOGI PENELITIAN

3.1 Kerangka kerja Penelitian

Penelitian ini diawali dengan melakukan studi Pustaka untuk mendapatkan referensi yang mendukung seperti cloud computing, dasar kerja load balancing serta metode *Least Connection*. Lalu melakukan pembuatan topologi jaringan serta melakukan implementasi dan konfigurasi sistem dengan menggunakan metode *Least Connection* menggunakan HAProxy pada sistem cloud secara sederhana dengan bantuan virtual machine.

Tahapan berikutnya melakukan pengujian dengan menggunakan apache jmeter sebagai. Setelah mendapatkan hasil dari pengujian akan dilakukan analisa untuk melihat kinerja load balancing pada sistem cloud yang telah bangun, Untuk dapat memudahkan proses tersebut maka peneliti akan membuat diagram alir dari kerangka kerja penelitian yang dapat dilihat pada gambar 3.1.



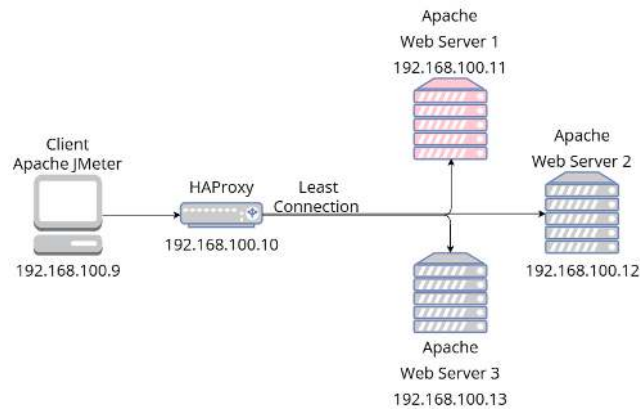
Gambar 3.1 Kerangka kerja Penelitian

3.2 Topologi Jaringan

Topologi jaringan memiliki peran penting dalam proses implementasi sistem cloud, khususnya dalam penerapan load balancing. Pada penelitian ini, topologi jaringan dirancang untuk mendukung pengujian performa metode *Least Connection* menggunakan HAProxy. Topologi ini meliputi seluruh komponen yang diperlukan, mulai dari client, load balancer dan server backend.

Terdiri dari lima mesin virtual yang dijalankan pada lingkungan VMware, yaitu, satu server client sebagai pengirim request (dengan Apache JMeter), satu server HAProxy sebagai load balancer, dan tiga server backend yang menjalankan Apache Web Server.

Konfigurasi ini dirancang untuk mengamati proses distribusi beban oleh load balancer dengan jumlah koneksi aktif terkini. Dari rancangan tersebut diperoleh hasil pengujian yang relevan terhadap performa sistem pada berbagai kondisi beban. Gambar topologi jaringan dapat dilihat pada gambar 3.2

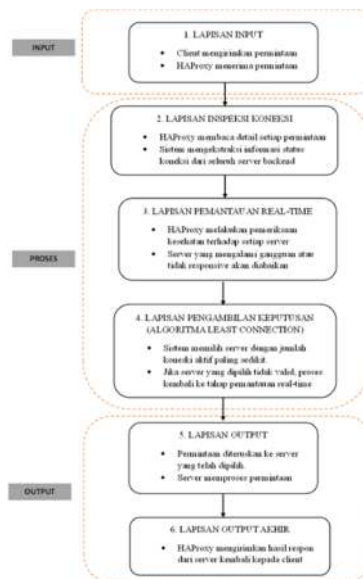


Gambar 3.2 Topologi Jaringan

3.3 Framework Cara Kerja Metode Least Connection

Metode Least Connection yaitu dengan menyalurkan permintaan pengguna ke server backend yang memiliki jumlah koneksi aktif paling sedikit. Metode ini sangat dinamis dan efisien dibandingkan metode yang ada pada load balancing, karena metode Least Connection mempertimbangkan kondisi nyata saat permintaan terjadi.

Dalam implementasinya, Client mengirim permintaan ke HAProxy yang bertindak sebagai load balancer, HAProxy kemudian memeriksa jumlah koneksi aktif pada masing-masing server backend. Permintaan akan diteruskan ke server dengan jumlah koneksi aktif paling sedikit. Contohnya, jika server 1 memiliki 6 koneksi aktif, server 2 memiliki 8 koneksi aktif, dan server 3 memiliki 4 koneksi aktif, maka HAProxy akan mengarahkan permintaan server 3. Proses ini akan dilakukan secara terus-menerus selama sistem berjalan. Jika salah satu server tidak aktif atau gagal merespons, HAProxy secara otomatis akan menghindari pengiriman permintaan ke server tersebut. Dalam penelitian ini, jika hanya satu server backend yang mati, maka proses load balancing tetap berjalan menggunakan dua server yang masih aktif. Namun, jika semua server backend mati, maka proses load balancing gagal dan memerlukan intervensi manual untuk pemulihan sistem. Cara Kerja Metode Least Connection yang dapat dilihat pada gambar 3.3



Gambar 3.3 Framework Cara Kerja Metode Least Connection

3.4 Pengujian

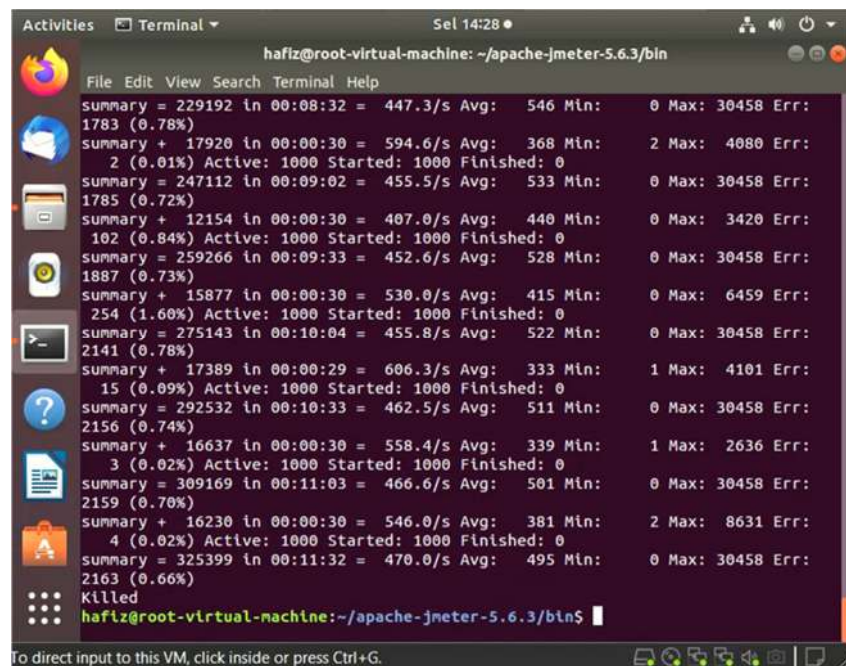
Pengujian dilakukan dengan melakukan percobaan stress testing menggunakan Apache J-Meter, di mana sejumlah thread user akan dikirimkan melalui HAProxy dan diteruskan ke web server. Jumlah thread user akan terus ditingkatkan secara bertahap hingga sistem tidak mampu lagi menampung permintaan secara optimal. Berikut skenario percobaan stress testing yang digunakan dalam penelitian, yang dapat dilihat pada tabel 3.1.

Tabel 3.1 Skenario Percobaan Stress Testing

No.	Jumlah Thread User	Ramp-Up (detik)	Duration (detik)	Start delay (detik)	Loop Count
1	10	10	60	2	Infite
2	100	5	180	2	Infite
3	300	3	300	2	Infite
4	500	2	480	2	Infite
5	750	1	600	2	Infite
6	1.000	1	900	2	Infite

Berdasarkan tabel 3.1, pengujian dilakukan secara bertahap dengan dimulai dari skenario pengujian ringan dengan jumlah thread user kecil, yaitu 10 thread user. Pengujian awal menggunakan ramp up yang cukup besar yaitu 10 detik, dengan durasi selama 60 detik, start delay selama 2 detik dan loop count diset tak hingga, agar thread user mengirim permintaan sampai waktu durasi.

Pada tahap-tahap berikutnya, jumlah thread user dan durasi pengujian ditingkatkan, sementara nilai ramp up dikurangi untuk mempercepat proses aktivitas user. Tahap akhir merupakan pengujian ekstrem (stress testing), dengan jumlah user maksimum, yaitu 1.000 thread user. Jika selama proses pengujian terjadi kondisi killed (berhenti secara paksa) pada Apache J-Meter, hal tersebut menunjukkan bahwa web server telah mencapai batas maksimum kapasitasnya. Kondisi tersebut dapat dilihat pada gambar 3.4 dibawah ini.



Gambar 3.4 Log Pengujian Thread User 1.000 Terjadinya Killed

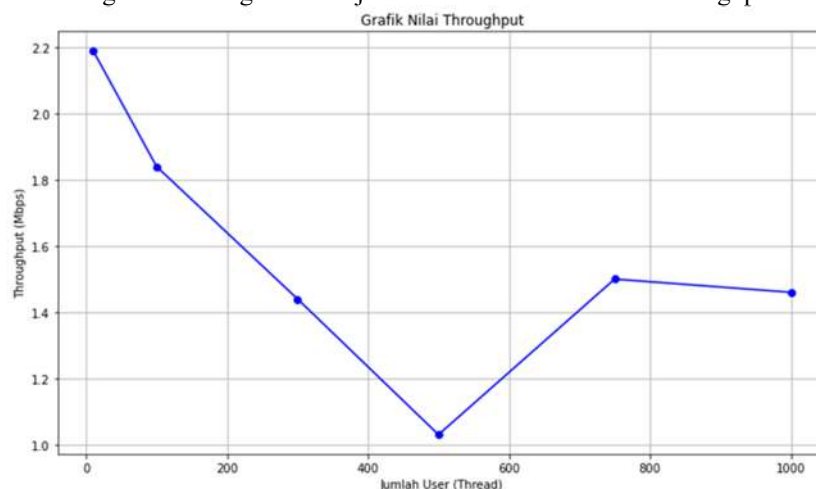
4. HASIL PENELITIAN

4.1 Analisa Throughput

Throughput merupakan parameter QoS yang digunakan untuk mengukur total jumlah permintaan yang berhasil di proses sistem dan durasi pengujian. Parameter ini penting untuk dapat menilai performa sistem, terutama dalam kondisi beban tinggi atau pada saat stress testing. Pengukuran Throughput dilakukan untuk mengetahui kapasitas maksimal sistem dalam melayani permintaan user.

Hasil nilai throughput, dapat dilihat nilai throughput tertinggi terjadi pada pengujian dengan 10 thread user, yaitu 2,19 Mbps. Dengan menambahkan jumlah thread user, nilai throughput mengalami penurunan, khususnya pada pengujian dengan 500 thread user, yaitu 1,03 Mbps, ini menunjukkan adanya beban berlebih pada server saat melakukan pengujian dengan 500 thread user.

Namun, pada 750 thread user nilai throughput meningkat kembali mengalami peningkatan yaitu 1,50 Mbps, kemungkinan disebabkan oleh optimasi alokasi beban oleh HAProxy. Pada 1.000 thread user terjadi penurunan kembali nilai throughput yaitu 1,46 Mbps, ini menunjukkan bahwa sistem telah mendekati batas optimalnya. Berikut ini adalah grafik hubungan antara jumlah thread user dan nilai throughput.

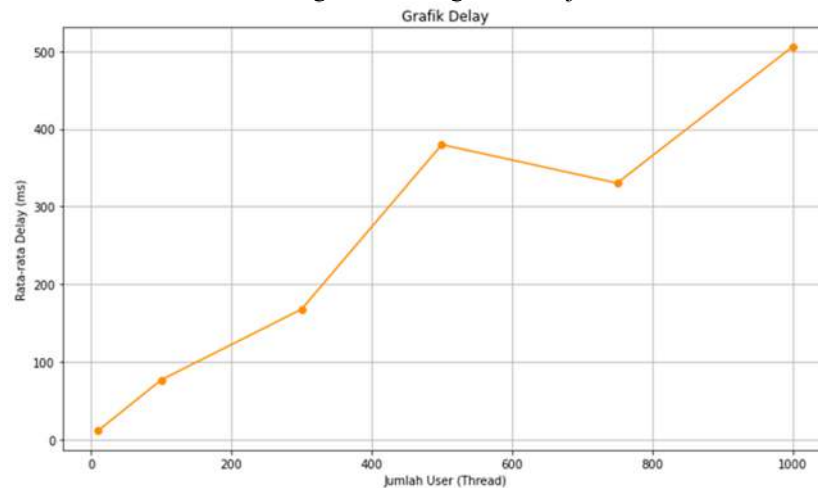


Gambar 4.1 Grafik Nilai Throughput

4.2 Analisa Delay

Delay merupakan waktu yang dibutuhkan untuk mengirim paket dari client ke server dan menerima respons kembali. Nilai delay didapatkan dari waktu tunda dari setiap permintaan (request) yang berhasil dikirim dan diterima. Semakin kecil nilai delay, maka semakin cepat waktu tanggap dari server.

Hasil delay, dapat dilihat bahwa nilai delay rata-rata meningkat dengan bertambahnya jumlah thread user. Pada 10 thread user delay rata-rata sebesar 11,26 ms, pada 750 thread user delay rata-rata lebih kecil dibandingkan dengan 500 thread user yang mencapai 379,75 ms, hal tersebut menunjukkan adanya efisiensi yang tercipta akibat distribusi beban yang lebih merata dan optimal oleh HAProxy melalui metode Least Connection. Peningkatan paling besar terjadi pada 1.000 thread user, ini menunjukkan bahwa semakin tinggi jumlah permintaan (request) yang dikirimkan, maka semakin lama juga waktu yang dibutuhkan sistem untuk merespons, akibatnya beban semakin berat pada web server. Berikut ini adalah grafik hubungan antara jumlah thread user dan delay.

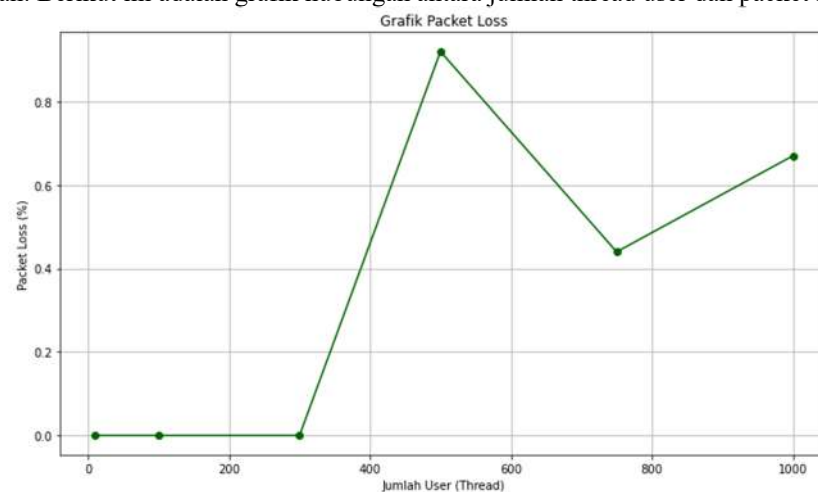


Gambar 4.2 Grafik Nilai Delay

4.3 Analisa Packet Loss

Packet loss merupakan parameter QoS yang mengukur jumlah paket data yang hilang selama proses transmisi dari client ke server. Paket yang hilang artinya tidak berhasil mencapai tujuan serta tidak mendapat respons. Nilai packet loss yang tinggi dapat menurunkan kualitas layanan yang cukup signifikan.

Hasil packet loss, dapat dilihat bahwa paket yang dikirim oleh jumlah thread user 10, 100, dan 300 sesuai dengan paket yang diterima oleh sistem yaitu 0.00%, hal tersebut menunjukkan bahwa tidak ada paket yang hilang. Nilai packet loss mulai terlihat saat jumlah thread user 500, yaitu sebesar 0.92%. Hal ini menunjukkan bahwa sistem mulai kehilangan paket akibat beban yang berlebih pada web server. Meskipun pada jumlah thread user 750 dan 1.000 nilai packet loss sedikit menurun menjadi 0.44% dan 0.67%, hal ini masih dalam batas toleransi. Namun, hal ini menunjukkan bahwa performa sistem mulai terdegradasi saat menangani permintaan dalam jumlah besar secara bersamaan. Berikut ini adalah grafik hubungan antara jumlah thread user dan packet loss.

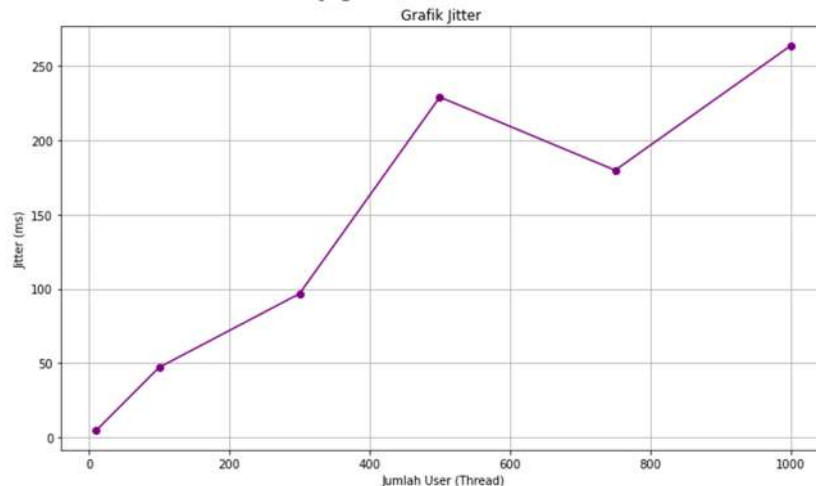


Gambar 4.3 Grafik Packet Loss

4.4 Analisa Jitter

Jitter merupakan variasi waktu delay antar pengiriman paket secara berurutan. Semakin besar nilai jitter, maka semakin tidak stabil pengiriman data. Pada web server pengaruh jitter lebih terbatas namun tetap relevan sebagai indikator kestabilan jaringan.

Hasil nilai jitter, dapat dilihat bahwa jitter meningkat dengan bertambahnya jumlah thread user. Nilai jitter terendah terjadi pada pengujian 10 thread user, yaitu 4,51ms. Nilai jitter tertinggi terjadi pada 1.000 thread user, yaitu 263,78ms. peningkatan nilai jitter mencerminkan adanya fluktuasi waktu delay yang semakin besar ketika jumlah permintaan (request) meningkat. Pada pengujian 750 thread user nilai jitter mengalami penurunan ,yaitu 179,93 ms dibandingkan dengan 500 thread user, yaitu 229,21 ms. Penurunan tersebut disebabkan oleh efisiensi distribusi permintaan ke server backend yang dilakukan oleh metode Least Connection, meskipun beban meningkat, distribusi menjadi lebih merata dan variasi delay antar paket menurun. Berikut ini adalah grafik hubungan antara jumlah thread user dan nilai throughput.



Gambar 4.4 Grafik Nilai Jitter

5. KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan tahapan pengujian yang dilakukan dengan menerapkan metode *Least Connection*. Didapatkan Kesimpulan pada penelitian ini adalah sebagai berikut:

1. Penerapan metode least connection terbukti efektif dalam mendistribusikan beban kerja ke server backend berdasarkan koneksi aktif paling sedikit. Setiap distribusi permintaan yang dikirim ke server terbagi secara merata dan efisien.
2. Parameter Quality of Service (QoS) menunjukkan performa sistem yang bervariasi sesuai jumlah thread user yang diuji. Salah satu contohnya adalah pada nilai Throughput tertinggi terjadi pada pengujian 10 thread user yaitu 2,19 Mbps. throughput menurun saat jumlah thread user meningkat, terutama pada pengujian 500 thread user hingga 1.000 thread user, menunjukkan adanya batas kemampuan layanan.
3. Jumlah thread user 1.000 merupakan titik maksimum dari pengujian. Terjadinya killed (berhenti secara paksa) menandakan server telah mencapai batas maksimum dan menghentikan pengiriman permintaan ke server.
4. Penggunaan HAProxy sebagai load balancer pada lingkungan virtual berbasis VMware dapat diimplementasikan secara efektif untuk simulasi load balancing pada cloud computing, serta memberikan hasil yang realistis terhadap pengujian performa distribusi beban.
5. Penelitian ini menunjukkan bahwa metode *Least Connection* juga cocok diterapkan pada sistem cloud berskala kecil hingga menengah yang membutuhkan distribusi beban secara adaptif terhadap kondisi koneksi yang dinamis.

5.2 Saran

Saran untuk penelitian mendatang adalah agar pengujian tidak hanya terbatas pada metode *Least Connection*, namun dapat menggunakan metode load balancing lain seperti, *Round Robin*, *Source Ip Hash*, dan *Weighted Least Connection* untuk memperoleh perbandingan performa dalam berbagai kondisi beban pada lingkungan cloud. Serta dapat menggunakan load balancer berbeda dari HAProxy. Disarankan juga untuk

menambahkan jumlah server backend dan memvariasikan spesifikasi virtual machine yang digunakan, sehingga pengaruh skalabilitas serta konfigurasi perangkat terhadap performa sistem lebih menyeluruh.

DAFTAR PUSTAKA

- [1] G. Xu, J. Pang, and X. Fu, "A load balancing model based on cloud partitioning for the public cloud," *Tsinghua Sci. Technol.*, vol. 18, no. 1, pp. 34–39, 2013, doi: 10.1109/TST.2013.6449405.
- [2] S. Afzal and G. Kavitha, "Load balancing in cloud computing – A hierarchical taxonomical classification," *J. Cloud Comput.*, vol. 8, no. 1, 2019, doi: 10.1186/s13677-019-0146-7.
- [3] I. Gusti, L. Putra, and E. Prisma, "Implementasi Load Balancing Pada Web Server Dengan Menggunakan Apache," *Manaj. Inform. Univ. Negeri Surabaya*, vol. 5, no. 2, pp. 117–125, 2016, [Online]. Available: <https://jurnalmahasiswa.unesa.ac.id/index.php/11/article/view/16413/14911>.
- [4] K. Ekmawan, "Analisa Implementasi Load Balancing Round Robin dan Least Connection Pada Web Server," *J. Teknol. Informasi, Komputer, dan Apl.*, vol. 3, no. 2, pp. 244–254, 2021, [Online]. Available: <http://jtika.if.unram.ac.id/index.php/JTIKA/>.
- [5] T. Wira Harjanti, H. Setiyani, and J. Trianto, "Load Balancing Analysis Using Round-Robin and Least-Connection Algorithms for Server Service Response Time," *Appl. Technol. Comput. Sci. J.*, vol. 5, no. 2, pp. 40–49, 2022.
- [6] G. Singh and K. Kaur, "An Improved Weighted Least Connection Scheduling Algorithm for Load Balancing in Web Cluster Systems," *Int. Res. J. Eng. Technol.*, vol. 5, pp. 1950–1955, 2018, [Online]. Available: www.irjet.net.
- [7] D. Choi, K. S. Chung, and J. Shon, "An improvement on the weighted least-connection scheduling algorithm for load balancing in web cluster systems," *Commun. Comput. Inf. Sci.*, vol. 121 CCIS, pp. 127–134, 2010, doi: 10.1007/978-3-642-17625-8_13.
- [8] L. H. Pramono, R. C. Buwono, and Y. G. Waskito, "Round-robin algorithm in HAProxy and nginx load balancing performance evaluation: A review," *2018 Int. Semin. Res. Inf. Technol. Intell. Syst. ISRITI 2018*, pp. 367–372, 2018, doi: 10.1109/ISRITI.2018.8864455.
- [9] I. Ali and N. Meghanathan, "Virtual Machines And Networks - Installation, Performance, Study, Advantages And Virtualization Options," *Int. J. Netw. Secur. Its Appl.*, vol. 3, no. 1, pp. 1–15, 2011, doi: 10.5121/ijnsa.2011.3101.
- [10] P. P. Singh, "Performance Evaluation of Linux Operating Systems," vol. 23, no. 2, 2023.
- [11] R. Gokilabharathi, P. Deepalakshmi, "Efficient Load Balancing to Enhance the Quality of Service (QoS) in SDN," *2nd Conf. Trends Electron. Informatics*, no. Icoei, pp. 1046–1051, 2018.
- [12] Indrianto, "Performance Testing On Web Information System Using Apache Jmeter," vol. 7, no. 2, pp. 138–149, 2023, doi: 10.22437/jiituj.v7i2.28440.